JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Handler

java.lang.Object
    java.util.logging.Handler

Direct Known Subclasses:

MemoryHandler, StreamHandler

```
public abstract class Handler
extends Object
```

A `Handler` object takes log messages from a `Logger` and exports them. It might for example, write them to a console or write them to a file, or send them to a network logging service, or forward them to an OS log, or whatever.

A `Handler` can be disabled by doing a `setLevel(Level.OFF)` and can be re-enabled by doing a `setLevel` with an appropriate level.

`Handler` classes typically use `LogManager` properties to set default values for the `Handler`'s `Filter`, `Formatter`, and `Level`. See the specific documentation for each concrete `Handler` class.

Thread Safety and Deadlock Risk in Handlers

Implementations of `Handler` should be thread-safe. Handlers are expected to be invoked concurrently from arbitrary threads. However, over-use of synchronization may result in unwanted thread contention, performance issues or even deadlocking.

In particular, subclasses should avoid acquiring locks around code which calls back to arbitrary user-supplied objects, especially during log record formatting. Holding a lock around any such callbacks creates a deadlock risk between logging code and user code.

As such, general purpose `Handler` subclasses should not synchronize their `publish(LogRecord)` methods, or call `super.publish()` while holding locks, since these are typically expected to need to process and format user-supplied arguments.

Since:

1.4

Constructor Summary

Constructors

Modifier	Constructor	Description

protected **Handler**()	Default constructor.

Method Summary

All Methods | Instance Methods

Modifier and Type	Method	Description
abstract void	**close**()	Close the Handler and free all associated resources.
abstract void	**flush**()	Flush any buffered output.
String	**getEncoding**()	Return the character encoding for this Handler.
ErrorManager	**getErrorManager**()	Retrieves the ErrorManager for this Handler.
Filter	**getFilter**()	Get the current Filter for this Handler.
Formatter	**getFormatter**()	Return the Formatter for this Handler.
Level	**getLevel**()	Get the log level specifying which messages will be logged by this Handler.
boolean	**isLoggable**(LogRecord record)	Check if this Handler would actually log a given LogRecord.
abstract void	**publish**(LogRecord record)	Publish a LogRecord.
protected void	**reportError**(String msg, Exception ex, int code)	Protected convenience method to report an error to this Handler's ErrorManager.
void		

	setEncoding (String encoding)	Set the character encoding used by this Handler.
void	**setErrorManager** (ErrorManager em)	Define an ErrorManager for this Handler.
void	**setFilter** (Filter newFilter)	Set a Filter to control output on this Handler.
void	**setFormatter** (Formatter newFormatter)	Set a Formatter.
void	**setLevel** (Level newLevel)	Set the log level specifying which message levels will be logged by this Handler.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

Handler

protected Handler()

Default constructor. The resulting Handler has a log level of Level.ALL, no Formatter, and no Filter. A default ErrorManager instance is installed as the ErrorManager.

Method Details

publish

public abstract void publish(LogRecord record)

Publish a LogRecord.

The logging request was made initially to a Logger object, which initialized the LogRecord and forwarded it here.

The Handler is responsible for formatting the message, when and if necessary. The formatting should include localization.

API Note:

To avoid the risk of deadlock, implementations of this method should avoid holding any locks while calling out to application code, such as the formatting of `LogRecord`.

Parameters:

`record` - description of the log event. A null record is silently ignored and is not published

flush

```
public abstract void flush()
```

Flush any buffered output.

close

```
public abstract void close()
```

Close the `Handler` and free all associated resources.

The close method will perform a `flush` and then close the `Handler`. After close has been called this `Handler` should no longer be used. Method calls may either be silently ignored or may throw runtime exceptions.

setFormatter

```
public void setFormatter(Formatter newFormatter)
```

Set a `Formatter`. This `Formatter` will be used to format `LogRecords` for this `Handler`.

Some `Handlers` may not use `Formatters`, in which case the `Formatter` will be remembered, but not used.

Parameters:

`newFormatter` - the `Formatter` to use (may not be null)

getFormatter

```
public Formatter getFormatter()
```

Return the `Formatter` for this `Handler`.

Returns:

the `Formatter` (may be null).

setEncoding

```
public void setEncoding(String encoding)
                 throws UnsupportedEncodingException
```

Set the character encoding used by this `Handler`.

The encoding should be set before any `LogRecords` are written to the `Handler`.

Parameters:

`encoding` - The name of a supported character encoding. May be null, to indicate the default platform encoding.

Throws:

`UnsupportedEncodingException` - if the named encoding is not supported.

getEncoding

```
public String getEncoding()
```

Return the character encoding for this `Handler`.

Returns:

The encoding name. May be null, which indicates the default encoding should be used.

setFilter

```
public void setFilter(Filter newFilter)
```

Set a `Filter` to control output on this `Handler`.

For each call of `publish` the `Handler` will call this `Filter` (if it is non-null) to check if the `LogRecord` should be published or discarded.

Parameters:

`newFilter` - a `Filter` object (may be null)

getFilter

```
public Filter getFilter()
```

Get the current `Filter` for this `Handler`.

Returns:

a `Filter` object (may be null)

setErrorManager

```
public void setErrorManager(ErrorManager em)
```

Define an ErrorManager for this Handler.

The ErrorManager's "error" method will be invoked if any errors occur while using this Handler.

Parameters:

`em` - the new ErrorManager

getErrorManager

```
public ErrorManager getErrorManager()
```

Retrieves the ErrorManager for this Handler.

Returns:

the ErrorManager for this Handler

reportError

```
protected void reportError(String msg,
                           Exception ex,
                           int code)
```

Protected convenience method to report an error to this Handler's ErrorManager.

Parameters:

`msg` - a descriptive string (may be null)

`ex` - an exception (may be null)

`code` - an error code defined in ErrorManager

setLevel

```
public void setLevel(Level newLevel)
```

Set the log level specifying which message levels will be logged by this `Handler`. Message levels lower than this value will be discarded.

The intention is to allow developers to turn on voluminous logging, but to limit the messages that are sent to certain `Handlers`.

Parameters:

`newLevel` - the new value for the log level

getLevel

```
public Level getLevel()
```

Get the log level specifying which messages will be logged by this `Handler`. Message levels lower than this level will be discarded.

Returns:

the level of messages being logged.

isLoggable

```
public boolean isLoggable(LogRecord record)
```

Check if this `Handler` would actually log a given `LogRecord`.

This method checks if the `LogRecord` has an appropriate `Level` and whether it satisfies any `Filter`. It also may make other `Handler` specific checks that might prevent a handler from logging the `LogRecord`. It will return false if the `LogRecord` is null.

Parameters:

`record` - a `LogRecord` (may be null).

Returns:

true if the `LogRecord` would be logged.

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.